Programming Assignment 2

-

Report

41052 Advanced Algorithms

Group Members:
Finn Varvell - 24917724
Clayton Cheung - 24505392
Xuan Dung Dao - 14603753

1. The problem and the algorithms.

The Knapsack problem is a classic NP-Hard resource allocation problem.
Given a list of n items with associated weights and values, and given a fixed knapsack capacity, maximise the sum of values of items in the knapsack without the total weight of the items exceeding the capacity.

This report focuses on the 0/1 variant of the problem. The 0/1 property allows items to be either taken as a whole or excluded. Items can't be divided into fractions, and multiple copies of the same item can't be taken.

This problem is NP-hard, meaning there is not currently a known polynomial-time algorithm for solving it optimally.
A brute force solution to this problem would involve checking every possible combination of items, ignoring those whose total weights are greater than the capacity, and returning the maximum total value. There are 2^n possible subsets of items, giving this solution an exponential time complexity that quickly makes it impractical with large sets of items.

The three alternative algorithms considered by this project to solve the problem are dynamic programming, FPTAS (an approximation scheme) and a greedy heuristic.

The dynamic programming approach finds an exact optimal solution to the problem. It does this by evaluating and storing the optimal choices for smaller subproblems (smaller capacity and number of items) and using the results to build up to the final optimal solution.
The running time depends on the number of items and on the capacity, making it a pseudo-polynomial algorithm of complexity O(nW), where n is the number of items and W is the capacity. The space complexity can be reduced to O(W), as only the current and previous row of storage need to be accessed at any time.

Fully Polynomial-Time Approximation Scheme (FPTAS) doesn't guarantee an exact optimal solution, but instead it provides a solution that is within a specified percentage of the optimal solution. FPTAS uses a parameter called epsilon to control the accuracy of the approximation. Epsilon is a number between 0 and 1, a larger epsilon returns a less accurate result but decreases the computational requirements and a smaller epsilon increases the accuracy but requires more computation.

The first step for FPTAS is to find the maximum feasible item value, finding the highest value item will be used to calculate an appropriate scaling factor. The scaling factor will then be calculated with the highest value item calculated as:

$$K = \frac{\varepsilon \times V_{max}}{n}$$

Where n is the number of items. From this calculation, it will determine how much the original item's value will be reduced. The item values will be scaled down by dividing the values by the scaling factor and rounding down to an integer, this reduces the range of values that the DP part of FPTAS needs to consider.

With the new scaled down item values, a 2D DP table is then used to store the minimum weight required to achieve a particular total scaled value. For each item the algorithm considers whether to include it or exclude it by comparing the minimum weight required for each possible scaled value. Excluding the item will keep the previous result and the minimum weight for each value will stay the same. Including the item will add the item's weight to the existing result, add the item's scaled value to the existing scaled value and will check if this gives a lower minimum weight. The algorithm compares these two options and will keep whichever result has a smaller weight.

Once all items have been processed, the highest scaled value that can be achieved without exceeding the knapsack capacity will be found. Using the DP table and iterating backwards through all the items, the algorithm will backtrack to find which items were included in the capacity and return the indices of those items.

The time complexity for FPTAS is $O(n^3/\varepsilon)$, FPTAS is polynomial in the number of items and $1/\varepsilon$ rather than being dependent on the capacity. This is because each item needs to be processed and additionally for each item every possible scaled value is checked in the 2D DP table.

The greedy algorithm prioritises items depending on their value density. Particularly, the value density is calculated as:

$$\text{Value Density} = \frac{\text{Item Value}}{\text{Item Weight}}$$

First, the algorithm checks if the input is valid. More specifically, the capacity cannot be negative, item weights need to be positive, item values cannot be negative, and item IDs must be unique. Subsequently, the algorithm copies the items into a different list and sorts them from highest to lowest value density. If two items have the same value density, the order is sorted from lowest to highest IDs. The algorithm considers each item after sorting that if the item is within the remaining capacity, it selects it and updates the total weight and value. If not, the algorithm skips that item and continues checking the remaining items. Eventually, the outcome contains the total value, total weight and IDs of the selected items. This approach is simple since it makes each decision based on the value density of the current item. However, the algorithm does not check other possible ways to choose items, as it only follows its first decision, so it might not find the best solution.

2. Theoretical comparison and predictions.

The three algorithms approach the 0/1 knapsack problem in different ways. Dynamic programming aims for correctness, always returning the optimal solution. The greedy algorithm prioritises speed, finding a good solution with a simple strategy, and FPTAS finds an approximate solution using a tolerance parameter while remaining in polynomial time.

2.1 Dynamic Programming

The dynamic programming implementation uses a two-dimensional table which stores the maximum achievable value for each combination of n (number of items) and c (capacity). There are two options for each item: Including it or excluding it. If the item is excluded, the value is the same as the cell above in the table, which is the maximum achievable value with the items other than this one at the same capacity. If the item is included, the value of that item is added to the maximum achievable value with the other items, with the weight of the item subtracted from the capacity. The maximum of including or excluding the item is the optimal value for each cell.
In a table, this looks like:

$$DP[i][c] = Max(DP[i-1, c], Value[i] + DP[i-1, c-Weight[i])$$

Items with a weight greater than the capacity are excluded, and the included items can be reconstructed afterward by doing a single backwards pass through the table.

The time complexity is thus O(nW), where n is the number of items, and W is the capacity. This is pseudo-polynomial time, as the time complexity also increases with the capacity, rather than just being based on the input.
If the problem is just asking for the maximum value and the selected items do not need to be reconstructed, the space complexity can be reduced to O(W), as only the previous row ever needs to be accessed and earlier ones can be discarded. For this experiment, we return the selected items as well as the total value and weight, and the full 2D table was used, keeping the space complexity at O(nW).

The advantage of this approach is that it is guaranteed to produce an optimal solution. However, its performance is dependent on the capacity, and inputs with increasingly large capacities will slow the algorithm down considerably. The other algorithms do not depend on the capacity, and so the relative performance of this algorithm is expected to be better in the tests with smaller capacities.

2.2 FPTAS

FPTAS is a compromise of the accuracy of exact DP and the speed of greedy. Solution accuracy is sacrificed in order to reduce the computational work required and unlike the dp approach the complexity is not dependent on the capacity. This means that FPTAS is a better approach for getting more accurate results for problems with higher capacities. The accuracy of the algorithm is controlled using epsilon where a lower epsilon will be more

accurate at the cost of more computation and a higher epsilon will be less accurate but require less computation.

In terms of comparison between the other two algorithms, FPTAS' main application will be for when the items and knapsack have a higher capacity since it avoids using a dp table reliant on the capacity. Compared to greedy, FPTAS should still be slower since FPTAS still needs to run DP. FPTAS compared to exact DP should be quicker depending on the input items and the capacities. When the capacities are higher, FPTAS should be much quicker and more efficient than exact DP since the complexity is not dependent on the capacities.

2.3 Greedy

The greedy algorithm has the time complexity of O(n log n), where n is the number of items. Firstly, the algorithm checks whether the capacity is valid or not, followed by the weights, values, and IDs of the items. An unordered set was used to avoid duplicate IDs because it checks for duplicate IDs without going through the whole list more than once. Thus, this check takes O(n) since each item is checked once. Later, the algorithm copies the original item list to run experiments on without changing the original items, taking O(n) time since it copies all items. Next, the algorithm sorts the copied list from highest to lowest density, leading to O(n log n) time. The algorithm then scans the sorted list to select items that fit within the remaining capacity. If the item can fit, the weight, value and ID are added to the outcome. If not, the item is skipped. This process takes O(n) time. Overall, the time complexity is:

$$O(n) + O(n) + O(n \log n) + O(n) = O(n \log n)$$

Furthermore, the space complexity of the greedy algorithm is O(n), where n is the number of items. Specifically, the algorithm copies the original item list for sorting and uses an unordered set to check duplicate IDs. Finally, the algorithm stores selected item IDs in a vector. For each task, it takes O(n) space, so the overall complexity is O(n). Nevertheless, the algorithm does not always achieve the best total value by choosing only the highest-density items. For instance,

ID	Weight	Value
1	1	2
2	10	19

In this example, the greedy algorithm will choose item 1 first since it has a higher value density. Therefore, the remaining capacity is now 9, which cannot fit item 2. As a result, the algorithm returns a total value of 2, while the best solution could be item 2 with a total value of 19. This demonstrates the greedy algorithm's weakness that it does not consider other options yet only follows its first best decision.

2.4 Predictions

Algorithm	Time Complexity	Space Complexity	Correctness
DP	O(nW)	O(nW)	Optimal Solution
FPTAS	$O(n^3/\varepsilon)$	$O(n^3/\varepsilon)$	Approximate (Guaranteed)
Greedy	O(n log n)	O(n)	No Guarantee

Before running the experiments, we expected the greedy algorithm to have the fastest runtime, especially for larger input sizes. As the selection process is simple, it should be able to handle many more items without slowing down compared to the other algorithms.
The dynamic programming is expected to perform well when capacity is low, and increase runtime significantly as it grows larger.
FPTAS is expected to be particularly useful compared to DP when the capacity is large, as its runtime is unaffected by the capacity.
The correctness of FPTAS and the greedy algorithm can be measured by comparing their values to the DP output, which is optimal. Reducing the epsilon of FPTAS should keep the solution closer to the optimal solution, at the cost of reducing runtime.

The performance of the greedy algorithm depends on the number of items since it needs to sort them by value density. Hence, the more items, the longer the running time. The running time of the greedy algorithm does not directly depend on the knapsack capacity like Exact DP, but capacity can affect item selection and the final total value. However, it may produce a lower total value because it does not explore other possibilities. Moreover, the greedy algorithm is expected to perform better when high-density items fit in the knapsack, but it may perform worse if those items exclude more high-value items.

The performance of FPTAS will be dependent on both the number of items and the chosen epsilon value. Similar to greedy, FPTAS does not depend on the capacity unlike exact DP however, may not sacrifice as much accuracy as greedy. FPTAS would be greatly dependent on the number of items since its time complexity is $O(n^3/\varepsilon)$ rather than greedy O(nlogn). This would mean that the running time of FPTAS should exponentially grow when dealing with a higher number of items. Thus, the trade-off for having more accuracy than the greedy approach without needing to rely on the item and knapsack capacity like exact DP. The running time will also increase when epsilon is reduced since the tradeoff for more accuracy will be a longer run time and computation.

3. Experimental methodology.

The inputs to the knapsack algorithms are a list of items with weights and values, and a knapsack capacity. Random knapsack instances were generated using C++'s random number generation library. The number of items (n) and capacity (W) were changed for each test. n random items were generated each time, assigning them random weights up to the capacity, and random values between 1 and 1000. A fixed seed was used so that repeating the benchmark would generate the same numbers, allowing it to be reproduced and different benchmarks to be compared.

Parameter	Value
Number of items (n)	100, 500, 1000, 5000, 10000
Knapsack capacity (W)	100, 1000, 10000
Item weight	Random between 1 and capacity
Item value	Random between 1 and 1000
FPTAS Epsilon	0.2, 0.5

Ten different inputs were generated for each combination of item count and capacity. Because all algorithms receive the same inputs, the differences in runtime and solution quality can be guaranteed to be due to the algorithms rather than the input.

The benchmark was implemented using a std::chrono::steady_clock to measure execution time. Each algorithm was executed on the same input instance 10 times, and the total elapsed time was divided by 10 to get the average execution time of each call.
Regarding the inputs for the runtime test, the assigned inputs are calculated with i representing the item index as:

- The weight: (i % 50) will generate weights between 0 and 49, then + 1 to ensure the weight is never zero
- The value: (i %100) will generate values between 0 and 99, then + 10 to ensure the value is from 10 to 109.
- The capacity is calculated as n / 2 so it can increase as the number of items increases. This lets the algorithm test different input sizes while using the same capacity rule.

Although these inputs demonstrate the performance as the number of items increases, they are limited to a repeating pattern, so they cannot produce other possibilities.
The benchmark writes the results to a CSV file for analysis, with the following columns:

- Number of items (n)
- Knapsack capacity (W)
- Run number (1-10, for each combination of n and W)
- Average execution time for each of the three algorithms
- Total value returned by each algorithm
- Epsilon used by FPTAS

The execution time includes all the operations inside the algorithms, but not the input generation or writing to the CSV. The algorithms were also run 10 times before the clock started for warm up to reduce some first run effects.

The same computer was used for all benchmarks to avoid differences caused by hardware and software.

FPTAS was only tested on the 100 and 500 item sets, as it got very slow very quickly and it became impractical to test on values of n greater than 1000.

For the greedy algorithm, I checked not only whether the selected IDs existed in the original input and whether every item was selected once, but also whether the total weight and value of the selected items matched the original inputs. In the test_greedy.cpp, for the greedy algorithm, several small fixed test cases were used, including an empty list of items, zero capacity, items with the same value density, and invalid item weights. In addition, I need to ensure that the total weight cannot exceed the knapsack capacity. In the test_greedy_random.cpp, regarding the inputs for the random tests, each test has 10 fixed items with the range of weights from 1 to 20, values from 1 to 100, and capacities from 1 to 50. The rationale behind these ranges is that they can test the algorithm with different inputs while keeping them simple. In the same file, to be more specific about the optimal solution using those inputs, I also tried to measure how close the greedy algorithm was to the optimal solution when it did not produce the optimal one, rather than only checking whether the result was optimal.The following calculation was used to compute this ratio.

$$Ratio = \frac{Greedy\ value}{Optimal\ value}$$

For each input size, we measured the running time over 10 batches to see whether the outcomes changed between batches. In each batch, the greedy algorithm ran 100 times, since a single run might be too fast to measure. Subsequently, the total time of each batch was divided by 100 again to find the average time for one run.

4. Results and analysis.

4.1 Execution Time

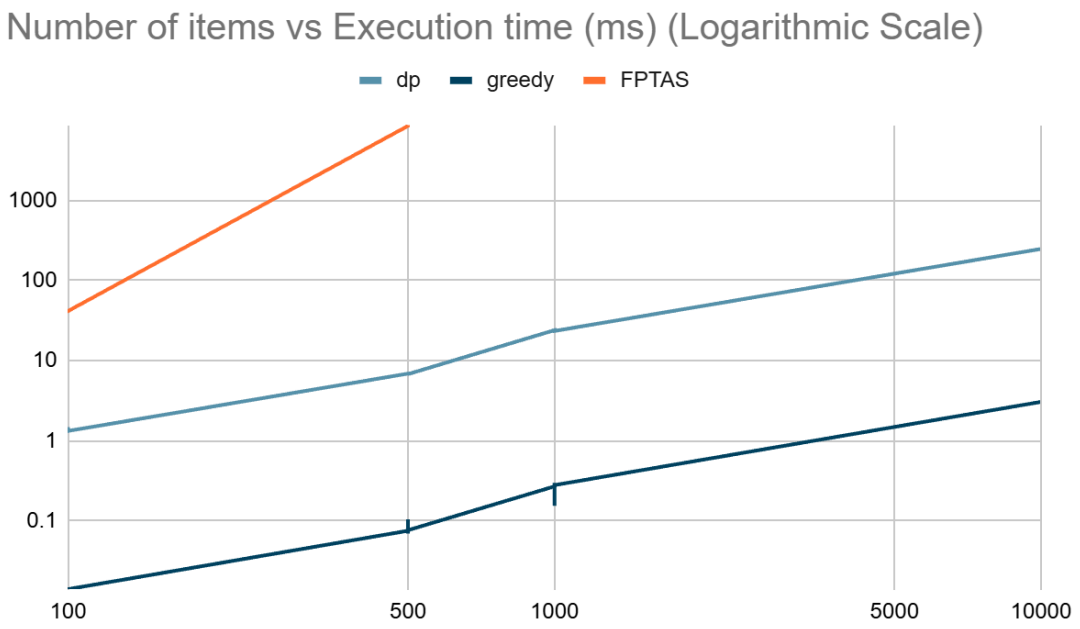
The results show a huge difference in execution time between greedy and dynamic programming. Greedy consistently had the shortest execution time across all tested input sizes and capacities. This is in line with expectations, because greedy only needs to sort the items and pass through the sorted list. Dynamic programming was much slower, as it evaluated every combination of item count and capacity.
The following shows comparisons in runtime for different items and capacities:

Items (n)	Capacity (W)	DP Time (ms)	Greedy Time (ms)	FPTAS Time (ms)
100	100	0.150	0.016	14.06
100	1000	1.37	0.014	13.63
100	10,000	14.75	0.015	13.98
500	100	0.728	0.087	2,662
500	1000	6.74	0.082	2,598
500	10,000	111.3	0.116	2,602
1000	100	2.47	0.281	Unrecorded
1000	1000	24.08	0.270	Unrecorded
1000	10,000	252.2	0.275	Unrecorded
10,000	100	24.86	3.08	Unrecorded
10,000	1000	243.1	3.05	Unrecorded
10,000	10,000	2,524.9	3.08	Unrecorded

FPTAS was averaged between 3 runs instead of 10, as it was taking much longer. When n was above 1000, the algorithm was too slow to run the benchmark.

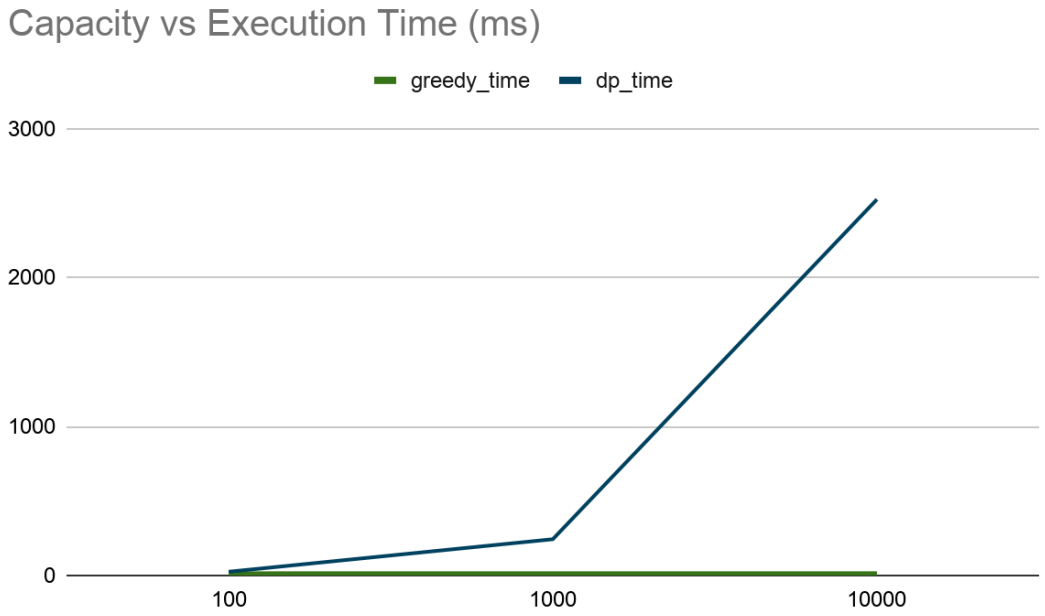
As expected, greedy was much faster on all inputs. FPTAS was the slowest of the algorithms in all tested instances. Although value scaling reduced the size of the DP problem, the additional overhead of the implementation caused it to take significantly longer.

The following graph shows a logarithmic scale of how the number of items changed the execution time. Capacity is kept at a constant 1000 for this graph.



One interesting difference is the effect of changing the capacity while keeping the number of items the same.

For 10,000 items, increasing the capacity from 100 to 10,000 increased the DP execution time from around 25ms to 2,525ms and an increase of approximately 101 times. This corresponds closely to the multiplication by 100 on the capacity. On the other hand, the greedy and FPTAS execution times stayed approximately constant across the same tests.



Number of items was kept at a constant 10,000 for the above graph

Increasing the number of items increases execution time for both algorithms. The effect is much greater in DP, as every new item is a new row of calculations across each capacity. DP becomes increasingly expensive when the item count and capacity get larger, making it less practical compared to greedy for large sets or capacities.

4.2 Solution Quality

DP will always return the optimal solution. FPTAS performed extremely well, while greedy occasionally produced a slightly lower total value.

This table shows some of the results for each algorithm, with FPTAS using an epsilon of 0.2

Items (n)	Capacity (W)	DP Result	Greedy Result	FPTAS Result
100	100	7823	7795	7823
100	1000	9558	9558	9558
100	10,000	9329	9314	9329
500	100	19383	19320	19383
500	1000	18651	18607	18651
500	10,000	15640	15640	15640

FPTAS achieved the optimal solution in almost every tested instance, even though the epsilon of 0.2 only guarantees a solution within 80% of the optimum.

Greedy performed well in these tests. It recorded the same value as DP in many cases, though occasionally the result was slightly lower. For example, one of the runs with 500 items and a capacity of 100 had greedy return a maximum value of 18,680, where the optimal value returned by DP was 18,836. This is only an approximately 0.83% difference, and most of the recorded results were within this range.

The results suggest that both FPTAS and the greedy algorithm can solve the problem close to optimally on the tested input sizes. FPTAS has a theoretical approximation guarantee of solution correctness according to the epsilon value, whereas greedy has no such guarantee. There are specific inputs that can be constructed where the greedy algorithms output is not as high. For example, the following test causes greedy to leave almost half of the knapsack empty, missing a much better solution.

```
Knapsack Test
 Weights {51, 50, 50} Values {102, 99, 99} Capacity: 100
Expected: 198
DP: 198
FPTAS: 198
Greedy: 102
```

For the greedy algorithm, the greedy algorithm passed all 100 random test cases, meaning that the valid items were selected without exceeding the knapsack capacity. Moreover, it also

produced the correct total weight and value for the selected items. The greedy algorithm was compared to brute force to check how close the outcomes were to the optimal solution.

Measurement	Outcome
Random tests passed	100/100
Optimal solutions found	69/100
Average ratio	98.26%
Lowest ratio	69.50%

This demonstrates that the greedy algorithm found 69 optimal solutions out of 100 test cases, and its average ratio was 98.26%, indicating that it often found values close to the optimal solutions. However, the algorithm did not find optimal solutions in other 31 cases, and its lowest ratio was 69.50%, meaning that some inputs produced worse results. This matches expectations, as it is expected to produce non-optimal solutions because it prioritises the first choice based on value density.

4.3 Overall Comparison

The strengths and weaknesses of each algorithm are clearly displayed when comparing the execution time with the correctness.

Greedy is consistently the fastest algorithm, but does not always find the optimal solution. Its main strength is its performance, and this is even more noticeable as the inputs get larger. This is because it only passes through the items a single time, and is not dependent on the capacity.

DP always provides an optimal solution, but its execution time increases significantly with both capacity and item count, making it less practical for larger inputs where capacity is high. Needing to go through every capacity for every item means that increasing both these numbers causes a huge increase to the number of calculations that need to be made.

FPTAS also provides almost optimal solutions, but is significantly slower than DP. The approximation guarantee was not shown as a practical speed advantage in these experiments.

We were unable to test FPTAS on larger input sizes, as the execution time was increasing too much, making it impractical. We cannot confidently determine how FPTAS would perform against the other algorithms with larger numbers of items and a much higher capacity.

5. What surprised you.

One of the most surprising results was when benchmarking FPTAS runtime against the DP and greedy algorithms. We expected FPTAS to trade some quality for a slightly better runtime, but FPTAS took significantly longer than DP in all tests.

The results showed that the theoretically useful improvement on the time complexity of the algorithm didn't necessarily translate into a better runtime on the tested inputs.

One explanation for this is the use of a two-dimensional DP table to tracethe selected items in the implementation of FPTAS, massively increasing the memory requirements and operations compared to an implementation that only returns an optimal value. The process of scaling the values and the number of columns in the DP table resulting from this could have caused a larger amount of computations.

DP is also fairly efficient for smaller capacities and smaller lists of items. If longer tests were performed with much larger capacities and numbers of items, FPTAS may have begun to outperform DP.

Another surprise was how well the greedy algorithm performed in terms of the solution quality. It found the exact optimal solution for a large number of the random inputs, and even when it did not, the difference was fairly small. This shows that value density is a useful selection strategy for a greedy algorithm with random inputs such as these experiments, though there may be constructed inputs where it would perform worse.

FPTAS also found almost perfect results in the tests performed. This was surprising as we were expecting it to be within 80% of the optimal value when using an epsilon of 0.2, but the results were consistently better than this. Overall FPTAS had a better solution quality than expected, but was much slower than expected compared to the other two algorithms for the inputs tested.
Furthermore, a surprising result was what would happen if I increased the number of random tests from 100 to 1000.
Initially, in 100 test cases, the results were:
Table 1. Results from 100 random tests

Measurement	**Outcome**
Random tests passed	100/100
Optimal solutions found	69/100
Average ratio	98.26%
Lowest ratio	69.50%

Subsequently, I expected that if I increased the test cases from 100 to 1000, the outcomes would be slightly different. Therefore, I increased the number of tests to 1000 while keeping the same input ranges and random seed.
Table 2. Results from 1000 random tests.

Measurement	Outcome
Random tests passed	1000/1000
Optimal solutions found	691/1000
Average ratio	97.70%
Lowest ratio	13.04%

As expected, the percentage of optimal solutions is nearly the same. Moreover, the average ratio also remained high, only slightly decreasing from 98.26% to 97.70%. Surprisingly, I was surprised by the lowest ratio, which dramatically dropped from 69.50% to 13.04%. This could happen because increasing the number of random tests could give more difficult inputs where the greedy algorithm makes poor decisions. This helps me understand why testing more inputs can reveal problems that smaller sets might miss.

6. What we'd do next.

There are many improvements that could be made on this project with additional time.

Firstly, there are many more inputs and edge cases that could have been investigated during the tests. While random inputs are fair, the current test had no correlation between the weights and values of the items, whereas a format where the weight of the item somewhat corresponds to its value could introduce differences in the correctness of the algorithms. Generating instances where many items have similar value densities could also help to find situations where the greedy algorithm performs especially well or poorly in terms of solution quality. Inputs where the weights of items could exceed the capacity shouldn't affect the output values, but may have also had an impact on the running time and could be considered in the inputs.

The differences in the algorithms when different outputs were expected could also be explored. For example, the current tests were on the algorithms returning a list of taken items, whereas in many instances of the knapsack problem only the maximum value is required, and this could affect the runtime of the three algorithms. Memory usage was also not benchmarked, and the space requirements between the three algorithms would be interesting to explore.

Regarding the greedy algorithm, what we could do next is to increase the number of random tests for the greedy algorithm. After seeing the significant drop from 69.50% to 13.04%, this made me wonder whether running more than 1000 tests would even reveal more difficult cases. Moreover, we also should try different ranges of weights, values, and capacities to see the impact of them to the greedy results.

If the FPTAS implementation could be optimised, or run for longer on a faster computer, the results for higher values of n could also be explored. FPTAS was slower in these experiments, but theoretically is more efficient than DP for extremely large capacities and values of n, and how the accuracy of its solutions changes for larger inputs could also be

explored. Testing all 3 algorithms on even larger inputs could also reveal larger differences in the correctness of FPTAS and the greedy algorithms as the problem gets more complicated.

7. Individual contributions.

Finn Varvell
I implemented the Dynamic Programming solution for the knapsack problem, including a 2D DP version that returns the selected items and a 1D version with a better space complexity that only returns the optimal value. I implemented these into the fixed benchmark, and designed and implemented the random benchmark. I ran all the tests on the random benchmark and stored the results in CSV files to analyse and compare for the report, constructing the tables and graphs with excel. I completed the outline for sections 1 and 2 of the report and the parts on dynamic programming, and filled out sections 3 to 6, outlining the experimental methodology and discussing the results. I recorded the dynamic programming and results section of the video.

Clayton Cheung
I completed my primary assigned algorithm which was the Fully Polynomial-Time Approximation Scheme (FPTAS). I implemented the FPTAS functions and code into the existing universal benchmark program for creating the results table. I also created the initial scaffold for the report to get us started with completing and filling out the report. In the report, I wrote all the information on the FPTAS in all the sections of the report and added additional information into sections 1 through to 6. To prepare for submission I proofread and prepared the entire document to ensure proper professional quality. Once we were ready, I prepared the video part of the submission by creating a video plan document where I planned and structured how the video will be produced. I also created the initial Google presentation for the supporting slides for the video. In the video, I recorded the introduction section of the video and the FPTAS explanation parts. I organised the group to record their sections and I edited the video together, including piecing the audio and slides into it.

Xuan Dung Dao
My responsibility is the implementation of the greedy algorithm for the 0/1 Knapsack problem in GreedyKnapsack.cpp and GreedyKnapsack.hpp. Moreover, the test_greedy.cpp was created to test the algorithm with fixed test cases and test_greedy_random.cpp was created to run random tests. In test_greedy_random.cpp, brute force was used to compare the outcomes of the greedy algorithm with the optimal solutions. Specifically, I compared 100 and 1000 random tests to observe the changes. In addition to the implementation, benchmark.cpp was created to measure the running time of the greedy algorithm using different input sizes; then I saved the outcomes in greedy_benchmark.csv and used Excel to create the chart visualising how the running time changed as the number of items increased. Regarding the benchmark for the Knapsack, I used the implementation of the separate benchmark of the greedy algorithm for the core of the full benchmark. Eventually, I contributed to the report by writing the sections on implementation, complexity, running time, testing methods, and limitations of the greedy algorithms. Moreover, the comparison between 100 and 1000 random tests are mine in the “What surprised you” part.

8. AI use.

Finn Varvell

ChatGPT was used when first understanding the algorithms to solve the problem to get a clearer understanding of the difference between the three algorithms before coding. After coding the DP solution, it was given to ChatGPT to find bugs and suggest optimisations. Gemini (On Google search) was also used when searching for c++ implementations of writing to csv and generating random numbers when working on the random benchmark. It was also used to help with the structure of the report, suggesting what comparisons would be useful for analysis. and drafting each section, which was then used as a reference for writing the report.

One example of when the AI was wrong was when explaining the logic behind the dynamic programming solution when discussing the code. ChatGPT suggested defaulting the amount when taking it to -1 (so that ignoring would always be chosen otherwise.) However, when the item could fit, it suggested adding the value of the item and also adding the DP from the previous row with capacity decreased by weight. This was the correct logic, however the first value was added instead of set, so the -1 was still included and the values ended up being 1 less than they should have been. ChatGPT later saw this when debugging the code and suggested defaulting at 0 or using = instead of +=.

ChatGPT also suggested running the tests using the same inputs ten times when initially discussing the random benchmark. This meant that each test ran 10 times into different columns in excel, in addition to the 10 times the algorithm ran for each test. Each column had the result from the final test and the time averaged from all 10 tests. Because the time was already averaged and the result did not change for the same input, this ended up with 10 almost identical columns, which the AI stated when questioned was not actually as helpful as randomising the number each time and only filling one column with the average time of the calls.

I also used AI to get an explanation of how FPTAS worked when running the benchmark, showing it the code. While this allowed me to get a general overview of the algorithm, it did not necessarily help me understand the specifics of the implementation. If this was not available we may have had more discussions as a group, which could lead to a better understanding of each other's algorithms.
Otherwise, AI was not used as a group.

Clayton Cheung

I used ChatGPT throughout the development of my FPTAS implementation to help understand the algorithm and fully complete the code implementation. I used it as a learning tool, asking for step-by-step explanations of concepts such as value scaling, the scaling factor (K), and the dynamic programming table. I also used it to help identify errors in my C++ implementation and understand why certain approaches worked or failed. During the report and presentation preparation, I used AI to additionally explain the theoretical time complexity on top of my own study and research. I checked the suggestions against my implementation, other research, assignment requirements and experimental results rather than accepting generated answers without verification. I used other resources I found online and other AI models such as Google AI to learn and further familiarise myself with every

detail of the FPTAS knapsack algorithm. After learning from a number of different sources I could then cross reference and verify my own understanding of the FPTAS Knapsack implementation rather than only trusting ChatGPT.

One main example that AI was misleading was that when I was doing my FPTAS implementation and asking for any knowledge gaps that I had, it seemed like ChatGPT believed that FPTAS would be faster than exact DP. I was misled to believe that with larger data sets and capacities FPTAS would be faster than exact DP. However, once I studied the time complexity of my FPTAS implementation and conducted additional research I found that FPTAS is exponentially dependent on the number of items. Thus, my FPTAS implementation would take increasingly longer on a higher number of items. Additionally, after conducting the benchmarking tests, this was solidified since FPTAS was extremely slow to the point where once we reached 1000 items FPTAS was too slow to finish.

Xuan Dung Dao

For the greedy algorithm, I used ChatGPT during this project to give me some suggestions and improve my code. For the greedy implementation in GreedyKnapsack.cpp and GreedyKnapsack.hpp, I wrote most of the implementation myself. However, I asked ChatGPT to help me improve some problems in my Assignment 1, such as handling overflow and invalid inputs using “throw”. Moreover, I also asked it to rename some variables to make my code easier to understand. For test_greedy.cpp, I wrote most of the tests myself and asked AI to suggest some fixed test cases to check the performance and explain how to use try/catch to test invalid inputs. I used AI for test_greedy_random.cpp the most. AI helped me use brute force to find the best possible value by checking different ways to reach the optimal solution. Moreover, AI also helped me calculate the ratio to check how close the greedy algorithms are to the optimal solutions. It also suggested a suitable range for each input to generate random tests. For the report, I asked ChatGPT to suggest a difficult example where the greedy algorithm would fail and improvements for the writing. Finally, I used it to clean up my code. Regarding the problems with AI, AI initially suggested me to compare the results for 100 random tests with the specific difficult inputs for the “What surprised me” section, but I realised this is unhelpful since the specific difficult inputs were made to fail the greedy algorithm, so there is no surprise, so I came up with the idea of increasing the random tests to 1000. In test_greedy.cpp, AI suggested me to focus too much on handling invalid inputs rather than focusing on other tests. I found this misleading to the centre of the assignment, so when I realised the performance tests were more important, I added more tests.

9. Individual Reflections

Finn Varvell

I programmed the Dynamic Programming solution to the knapsack problem, which goes through every combination of number of items and capacity to find the optimal solution to the problem, storing the values to help calculate future ones. The time complexity was therefore O(nW), which is the number of items multiplied by the capacity. The runtime is therefore not always manageable when the number of items and the capacity gets high, and the worst case happens when both are large. Increasing capacity especially showed the weakness of this algorithm compared to the other two, as their runtime was unaffected by the capacity, and this became clear when analysing the results from the benchmark.

The content covered in week 2 on 2D dynamic programming was very helpful for coding this algorithm, as I was able to easily draw out the table and visualise how each cell was the optimal value for that specific combination of items and capacity. I then realised that only the last two rows were being used in the solution, and made a different version that just stores the current and previous row, reducing the space complexity to O(W), which is something I did not do earlier in the subject. However, when it was time to benchmark I noticed that the other algorithms from my groupmates were returning lists of items chosen rather than just max value, so I returned to the first version as the 2D table could be easily used to reconstruct the chosen items, making the comparison more fair. According to ChatGPT, it is also possible to keep both the O(W) space complexity while reconstructing the items using some additional bookkeeping, so I would look into this if I were to do the implementation again from scratch.

I did not fully understand what Pseudo-Polynomial time meant as opposed to polynomial time before doing this implementation, despite covering it briefly in week 11. Even though O(nW) seems fairly small compared to some larger powers of n in polynomial time algorithms, the fact that it is scaling with respect to the magnitude of the input rather than the bit-length of the input means the runtime can increase exponentially according to the size of the input as the capacity gets larger, and I have a much better understanding of this difference and what makes a problem NP hard after analysing the results from the three algorithms.

Clayton Cheung

I programmed the Fully Polynomial-Time Approximation Scheme (FPTAS) approach to the knapsack problem. My FPTAS implementation uses a scaling factor which is dependent on the highest value item in the input, which is then used to scale down all the values. After scaling the implementation uses a 2D DP table to then store the minimum weight needed to achieve each scaled value. Once the DP table is filled, the program then iterates backwards through the list of items in order to determine which item was included or excluded and builds the taken list of items.

The worst case for the FPTAS implementation is when the number of items is high. Per the benchmarking results when the number of items reached 1000 we were unable to continue the testing as the run time became too long. This is because of the time complexity of my FPTAS implementation is $O(n^3/\varepsilon)$ which exponentially increases with higher n. The input

which triggers the worst case would be an input with over 600 to 1000 items. Despite this, from our testing the running time is not affected by the changes in capacity, which is expected since capacity is not a variable in the time complexity.

The hardest decision I encountered when implementing my FPTAS algorithm was choosing to use a 2D DP table to store the minimum weights needed for each scaled value. Using this 2D DP table to store the minimum weights allows me to then backtrack through the list of items to find which items were taken by returning the item indices. Using a 1D DP table could reduce the memory usage of my algorithm, however would increase difficulty in reconstructing the list of items that were selected. The algorithm also uses i - 1 when iterating through the list of items to decide whether or not to include or exclude the item. Since it uses the previous item in the list when considering the current item, it ensures that each item is only used once rather than accidentally duplicating items.

If I were to do my FPTAS implementation again from scratch I would mainly focus on looking for a more efficient choice over the 2D DP table. Since my current implementation of FPTAS uses a 2D DP table, once the table is filled it takes a lot of memory and run time when the number of items is increased. Instead, using a 1D DP table may be much more efficient, but I would need to develop a different way to reconstruct the items from the scaled values.

When I initially started studying FPTAS and how it solves the knapsack problem, I didn't understand how it would be different from exact DP since both algorithms use DP. At first I didn't initially understand the importance of scaling the values. After implementing, testing and comparing the two algorithms, I understood how scaling the values affects the running time of FPTAS. Where the exact DP depends on the capacity which gives it pseudo-polynomial time where if the capacity increases then the DP table will increase. In FPTAS, it scales the capacities down depending on the highest value. This sacrifices some solution accuracy in order to reduce the size of the capacities where rebuilding the taken items from the scaled values reduces accuracy.

Xuan Dung Dao

The worst case for my greedy algorithm is when the algorithm chooses the item with the highest value density, leading to a lower total value than the optimal solution. The greedy algorithm could perform poorly when it selects an item with higher value density, leaving sufficient capacity for another item with a much higher value. For example, in my test with a capacity of 10, the greedy algorithm returned a value of 2, while the optimal value was 19.

ID	Weight	Value
1	1	2
2	10	19

Initially, I forgot this case while implementing the greedy algorithm. However, I then remembered that the greedy algorithm does not always the optimal solutions. Therefore, I asked AI to generate a specific difficult case to confirm this. This example helped me

understand that the greedy algorithm could make poor decisions despite following its selection correctly. Moreover,

Measurement	Outcome
Random tests passed	100/100
Optimal solutions found	69/100
Average ratio	98.26%
Lowest ratio	69.50%

The fact that the greedy algorithm found only 69 optimal solutions out of 100 tests made me realise its weakness in the 0/1 knapsack problem. The hardest design decision was how to handle invalid input. Initially, I only implemented all the main functions of the greedy algorithms, yet it gave me some bugs when the random tests gave invalid inputs, such as negative capacity. Therefore, I asked AI to help me handle these invalid inputs using throw and try/catch. Moreover, when I tried to compare the greedy results with the optimal solutions, I struggled to implement the brute force approach, so I asked AI for help later.
If I could implement this algorithm again, I would initially handle the invalid inputs during the implementation, so later it would not frustrate me when running the algorithm; then I would focus on the tests with different inputs. For example, the fixed tests would be designed to check if the algorithm could run well. Next, I would use random tests with different numbers of cases to witness the weakness of the algorithm. For example, I only ran 100 test cases initially, but then when I ran again with 1000 test cases, the results surprised me with the significant drop in the lowest ratio, meaning that increasing the number of random test cases could create more difficult cases for the greedy algorithm to handle.
Before the implementation of the greedy algorithm, I understood that the greedy algorithm does not always give the optimal solution, but it even surprised me more when I calculated how close the greedy results were to the optimal solutions. The results helped me realise how poor the performance is when the greedy algorithm handles difficult cases. Moreover, I also learned how to handle invalid inputs better using throw and try/catch, and to use brute-force to find the best value for small inputs and compare it with the greedy approach using the ratio. This project helps me understand the essentials of correctness and solution quality in a heuristic algorithm.